

Assignment - 01

Name - Shinde Rohit Popatrao

Roll no - ~~263332118~~ 263332188

Div - 'B' Sid - Sy.

Subject - operating system

* Types of OS

1] Batch OS

Definition - Executes jobs in batches without user interaction

Advantages:

- High CPU utilization
- can process many jobs automatically
- Suitable for repetitive

Disadvantages:

- No direct user interaction
- long waiting time
- Difficult to debug errors.

2] Multiprogramming OS.

Defn - keeps multiples programs in memory and executes them one by one to keep the CPU busy.

Advantages

- Better CPU utilization
- Increases system efficiency
- Reduces CPU idle time

Disadvantages.

- complex memory management
- Difficult scheduling
- Requires more memory

Uses: - office computers, Software development, Data processing system

3] Multitasking OS

Defn:- Allows a user to run multiple application at the same time.

Advantages

- Improves productivity
- Save time
- Better user experience

Disadvantages

- Requires more RAM
- System may slow down many programs

4] Multiprocessing OS

Defn:- Uses two or more processors to execute task simultaneously

Advantages:-

- Fast processing
- Higher reliability
- Better performance

Disadvantages

- Expensive hardware
- Complex system design
- Higher power consumption

5] Real time OS (RTOS)

Defn:- process task within a fixed time limit

Advantages

- very fast response
- Highly reliable
- Suitable for critical system.

Disadvantages

- Expensive develop
- limited flexibility
- complex programming

Uses:- Air traffic control system, medical equipment, industrial automation.

6] Distributed OS

Defn: - manages multiple connected computers as a single system

Advantages

- Resource sharing
- High performance
- Better reliability

Disadvantages

- complex maintenance
- Network dependency
- Security challenges

uses: - cloud computing, Banking networks, university and research network.

7] Network OS (NOS)

Defn: - manages computers connected over a network

Advantages

- centralized management
- File, printer sharing

Disadvantages:

- Server dependency
- maintenance cost

Linux System call.

Definition of System call

A System call is a mechanism through which a user program request services from the operating system kernel. System calls provide an interface between user application and the OS for performing operations.

Types of System calls.

1] Process control System calls

These system calls are used to create, execute, terminate and manage process.

eg- "fork()", "execve()", "exit()", "wait()",

2] File management System call.

These system calls are used to create, open, read, write, close and manipulate files.

eg- "open()", "read()", "write()", "close()",
"lseek()", "stat()",

3] Device management System calls.

These system calls allow communication with hardware devices such as disks, printers, keyboards, network devices.

ex- "read()", "write()", "fcntl()",

4] Memory management system calls.

These system calls manage memory allocation, deallocation, and mapping between user process and memory.

ex- "mmap()", "brk()", "munmap()".

5] Information maintenance system calls.

These system calls obtain or modify system and process information.

ex- "getpid()", "nice()", "getpriority()", "setpriority()",

6] Communication system calls.

These system calls enable communication between processes using pipes, sockets, shared memory and message queues.

ex- "pipe()", "socket()", "send()", "recv()".

Definitions of the given system calls

1] "fork()"

→ create a new child process by duplicating the calling (parent) process

2] "vfork()"

→ create a child process that shares the parent's address space until it calls "execve()" or exits.

3] "clone()"

→ creates a new process or thread with shared resources depending on specified flags.

4] "execve()"

→ Replace the current process image with a new program

5] "exit()"

→ Terminates a process after performing standard cleanup operations.

6] "_exit()"

→ Immediately terminates a process without performing standard library cleanup.

7] "wait()"

→ Suspends the parent process until one of its child processes terminates

8] "waitpid()"

→ wait for a specific child process to terminated

9] "waitid()"

→ wait for state change in child process and provides detailed status information

10] "kill()"

→ send a signal to a process or process group

11] "getpid()"

→ Returns the process ID (PID) of the calling process

12] "getppid()"

→ Returns the parents process ID (PPID) of the calling process

13] "nice()"

→ changes the scheduling priority of process

14] "setpriority()"

→ sets the scheduling priority of a process, process group, or user.

15] "getpriority()"

→ Retrieves the scheduling priority of process

16] "open()"

→ opens an existing file or creates one if specified with appropriate flags

17] "create()"

→ creates a new file or truncates an existing file for writing

18] "close()"

→ close an open file descriptor

19] "read()"

→ Reads data from a file or device into a buffer

20] "write()"

→ writes data from a specified file offset without changing the current file

21] "pwrite()"

→ writes data to a specified file offset without changing the current file pointer

22] "fseek()"

→ changes the current position of file pointer

23] "lseek()"

→ Retrieves information about file using its pathname

24] "stat()"

→ Retrieves information about a file using file descriptor

25] "fstat ()"

→ Retrieves information about a symbolic link without following it.

26] "lstat ()"

→ checks whether a file exists and whether the process has permission to access it

27] "access ()"

→ Deletes a file for the file system.

28] "unlink ()"

→ changes the name or location of a file or directory

29] "mkdir ()"

→ create a new directory

30] "rmdir ()"

→ Removes an empty directory

31] "dup ()"

→ duplicates an existing file descriptor

32] "dup2 ()"

→ Duplicates a file descriptor to a specified descriptor number

33] "fcntl ()"

→ performs various operating of file descriptors such as duplication locking and changing flags.

35] "fsync ()"

→ writes all modified file data and metadata to permanent storage.

36] "fdatasync ()"

→ writes modified file data to disk but may delay writing unnecessary metadata

37] "truncate ()"

→ changes the size of a file using the pathname

38] "ftruncate ()"

→ changes the size of a file using its file descriptor.